

RECURSION / RECURSIVE FUNCTIONS

It is the process of calling a function itself.

Purpose:

Recursion allows the user to get results , without using loops. Due to this complexity of program is reduced.

Recursion reduce calling of function by the user.

By using recursion, we can control the function calling information or statements.

By using recursion, we can evaluate stack expressions.

Drawbacks:

They are slower than normal functions due to stack overlapping.

They can create stack over flow because of occupying more stack.

Recursion functions will create infinitive loops also.

Eg: 1

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
{
printf("Welcome to C\n");
main();
}
```

Note: This program causes infinitive loops.

Eg 2: Controlling the above program

```
#include<stdio.h>
#include<conio.h>

int  a=1; /* global variable*/

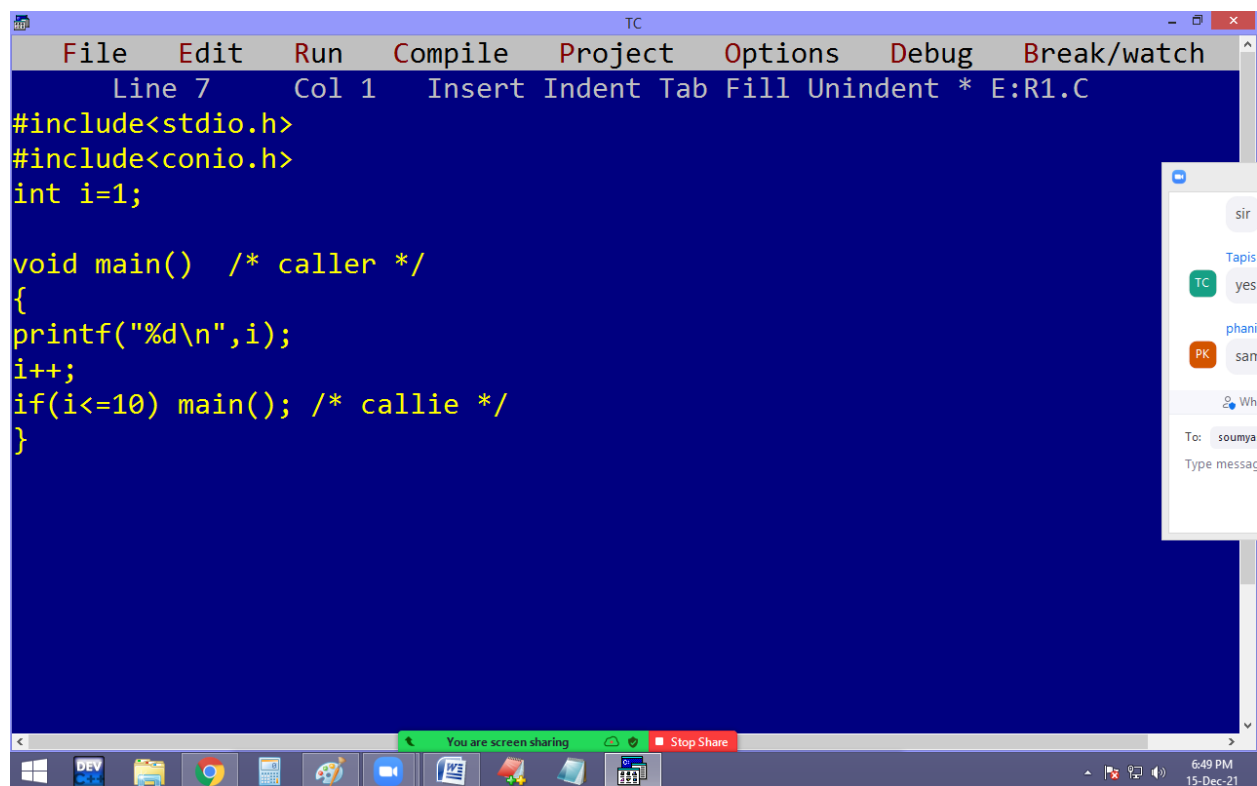
void main()
{
printf("Welcome to C\n");

a++;

if(a<=3) main();

getch();
}
```

Eg. printing 1..10 numbers using recursion.



The screenshot shows the Turbo C++ (TC) IDE with a blue background. The menu bar includes File, Edit, Run, Compile, Project, Options, Debug, and Break/watch. The status bar at the top indicates 'Line 7 Col 1 Insert Indent Tab Fill Unindent * E:R1.C'. The code in the editor is as follows:

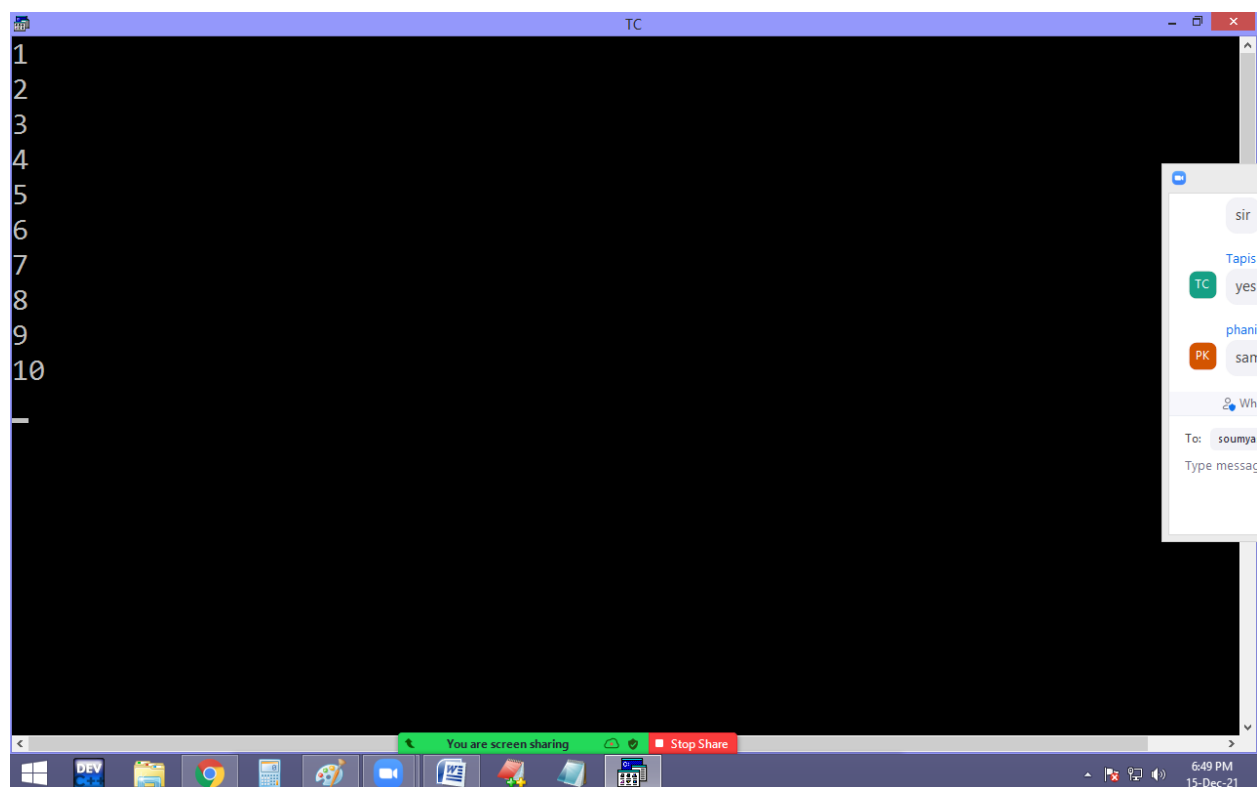
```
#include<stdio.h>
#include<conio.h>
int i=1;

void main() /* caller */
{
printf("%d\n",i);
i++;
if(i<=10) main(); /* callie */
}
```

On the right side, there is a chat window with the following messages:

- TC: yes
- phani: sarr
- Wh: soumya

The Windows taskbar at the bottom shows the time as 6:49 PM on 15-Dec-21. A green bar above the taskbar says 'You are screen sharing' with a 'Stop Share' button.



This screenshot shows the same Turbo C++ IDE after execution. The code area is now black, and the left margin shows line numbers 1 through 10. The chat window on the right remains the same. The Windows taskbar and the 'You are screen sharing' bar are also visible.

Eg: Finding factorial using recursion:

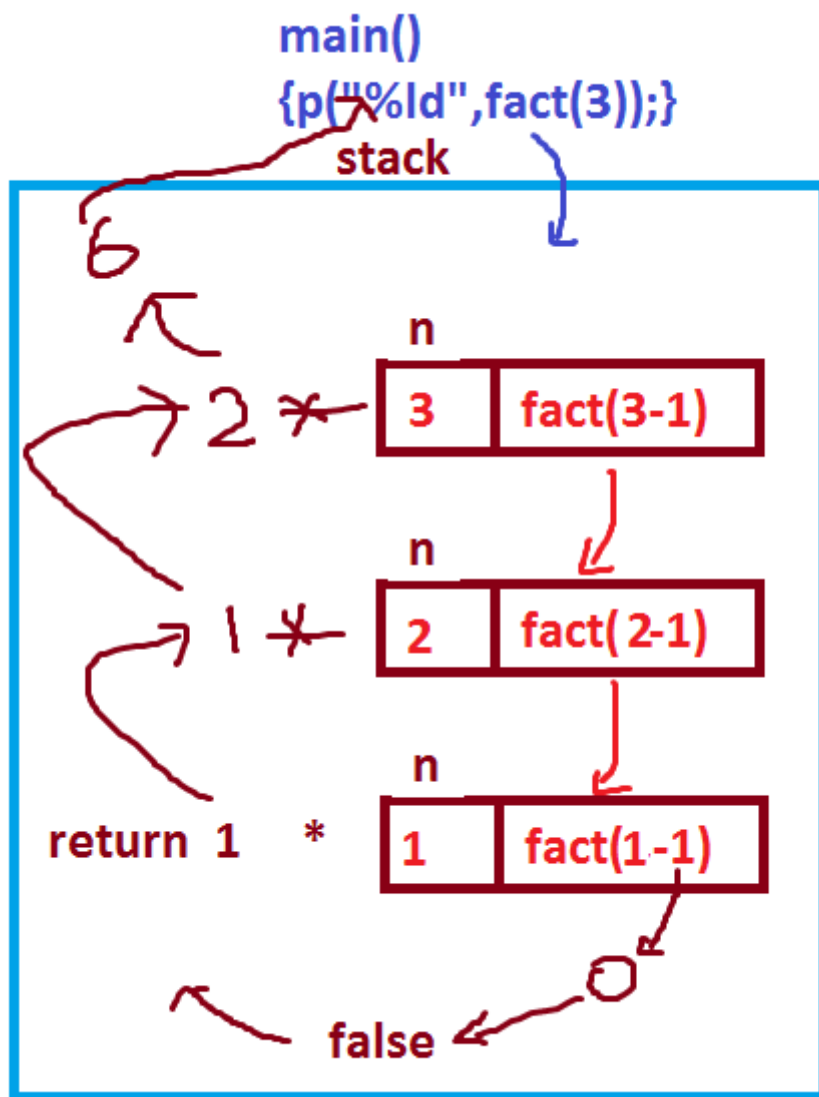
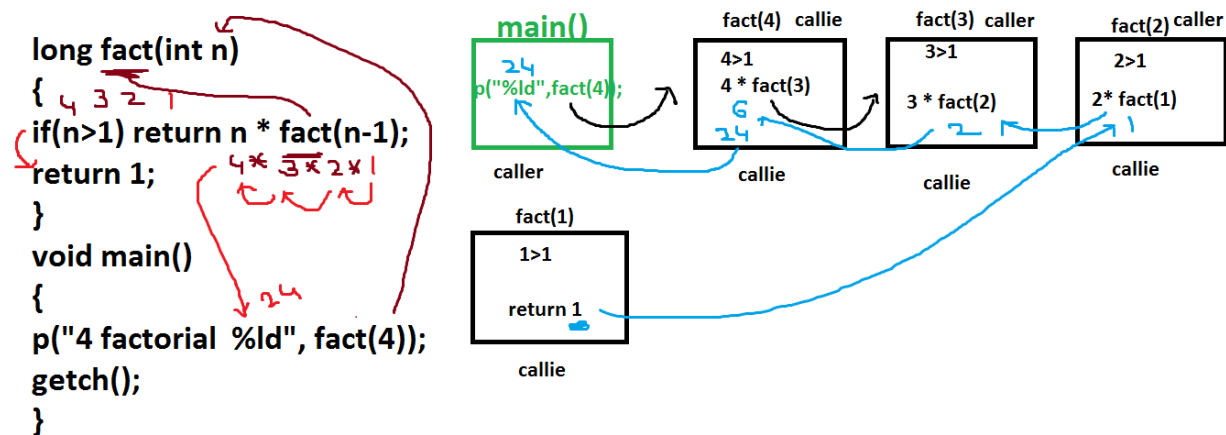
```
#include<stdio.h>
#include<conio.h>

long fact(int n)
{
    if(n!=0) return n * fact(n-1); else return 1;
}

void main()
{
    int n;
    clrscr();
    printf("Enter a no "); scanf("%d", &n);
    printf("%d Factorial = %ld", n, fact(n));
    getch();
}
```

O/P: Enter a no 5

5 Factorial = 120

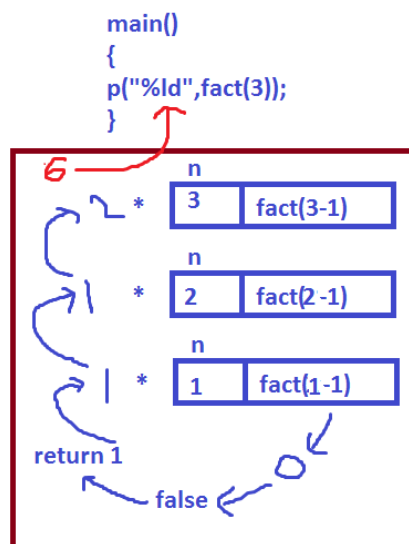


```

#include<stdio.h>
#include<conio.h>
long fact(int n)
{
if(n!=0) return n * fact(n-1);
else return 1;
}
void main() /* main is caller */
{
printf("3 factorial=%ld",fact(3));
getch();
}

```

6=3 * 2 * 1 * 1



Finding power using recursion:

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
long power(int b, int p)
```

```
{
```

```
if(p!=0) return b * power(b, p-1); else return 1;
```

```
}
```

```
void main()
```

```
{
```

```
int b,p;
```

```
printf("Enter base, power values ");
```

```
scanf("%d %d",&b,&p);

printf("%d ^ %d = %ld", b, p, power(b,p));

getch();

}
```

Output: Enter base, power values 2 5

2 ^ 5 = 32

```
#include<stdio.h>
#include<conio.h>
long power(int b, int p)
{
if(p!=0) return b * power(b,p-1);
else return 1;
}
void main() /* main is caller */
{
printf("2^3=%ld",power(2,3));
getch();
}
```

Handwritten notes on the code:

- For the recursive call `power(b, p-1)`, there are two vertical annotations: $\frac{p}{2}$ and $\frac{b}{2}$.
- Below these, the expression $2 = 2 \times 2 \times 2 \times 1$ is written, with an arrow pointing from the `power(2,3)` call in the code to the first '2' in the multiplication.

Eg : Finding digital sum using recursion

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
int s=0; /* global var*/
```

```
int dsum(long n)
```



```
{  
if(n!=0)  
{  
s=s+n%10;  
dsum(n/10);  
}  
return s;  
}  
  
void main()  
{  
long n;  
clrscr();  
printf("Enter a no");  
scanf("%ld",&n);  
printf("%ld digital sum = %d",n,dsum(n));  
getch();  
}
```

Output:

Enter a no: 123

123 digital sum = 6

```
#include<stdio.h>
#include<conio.h>
int dsum(long int n)
{
    static int s;
    if(n!=0)
    {
        s+=n%10; dsum(n/10);
    }
    return s;
}
void main() /* main is caller */
{
    printf("123 digital sum=%d",dsum(123));
    getch();
}
```

$$\frac{n}{10}$$

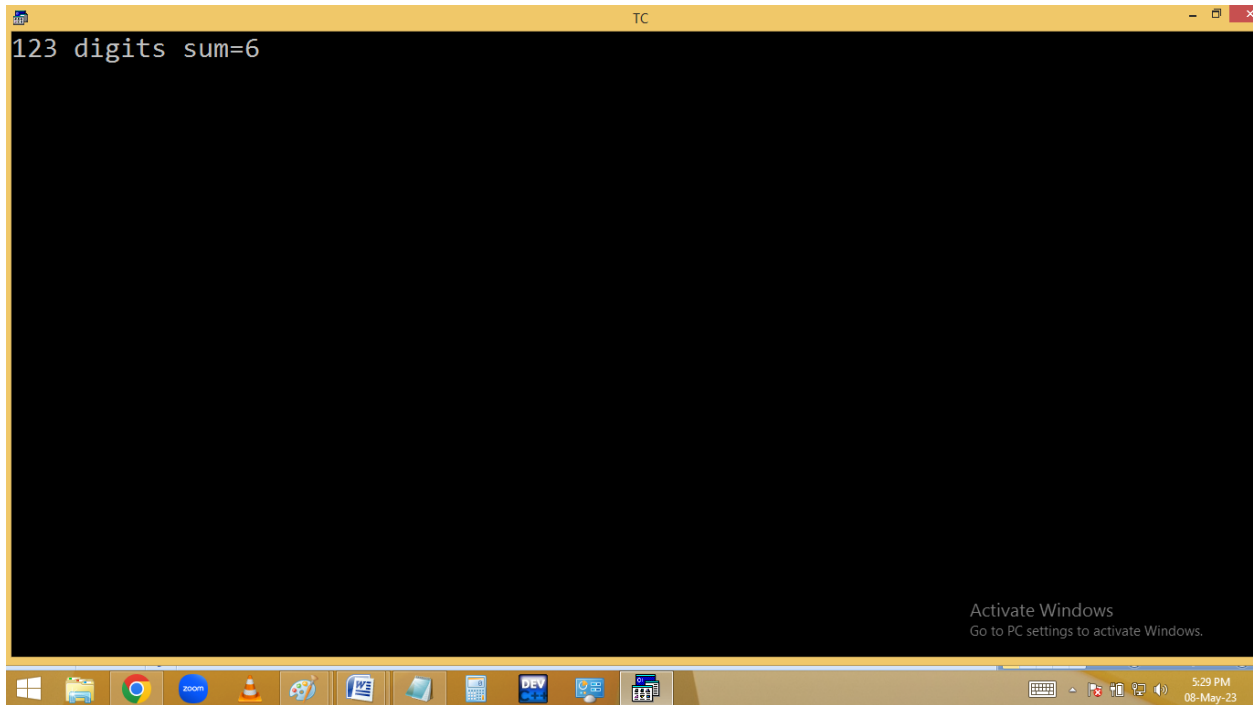
$$\frac{5}{0+3+2+1}$$

6

```
File Edit Run Compile Project Options Debug Break/watch
Line 6 Col 10 Insert Indent Tab Fill Unindent * E:4PM.C
#include<stdio.h>
#include<conio.h>
int dsum(long n)
{
    if(n!=0) return n%10 + dsum(n/10);
    return 0;_
}
void main()
{
    printf("123 digits sum=%d",dsum(123));
    getch();
}
```

Activate Windows
Go to PC settings to activate Windows.

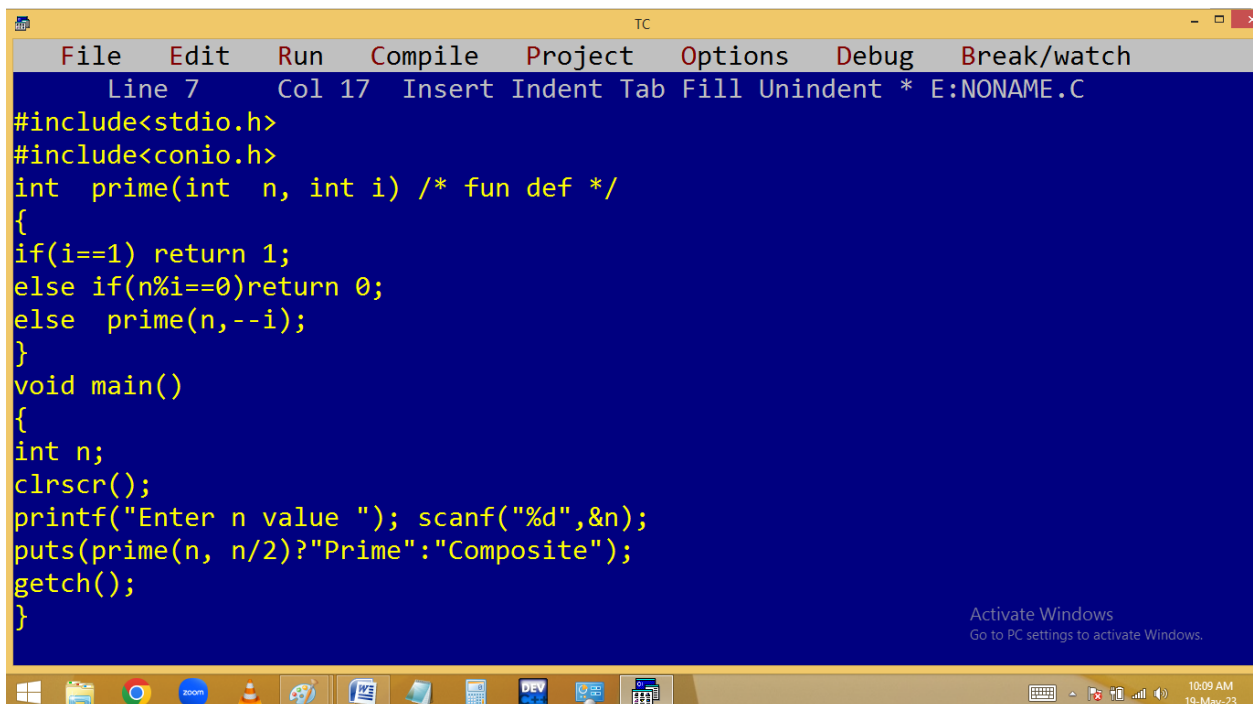
5:28 PM
08-May-23



```
123 digits sum=6
```

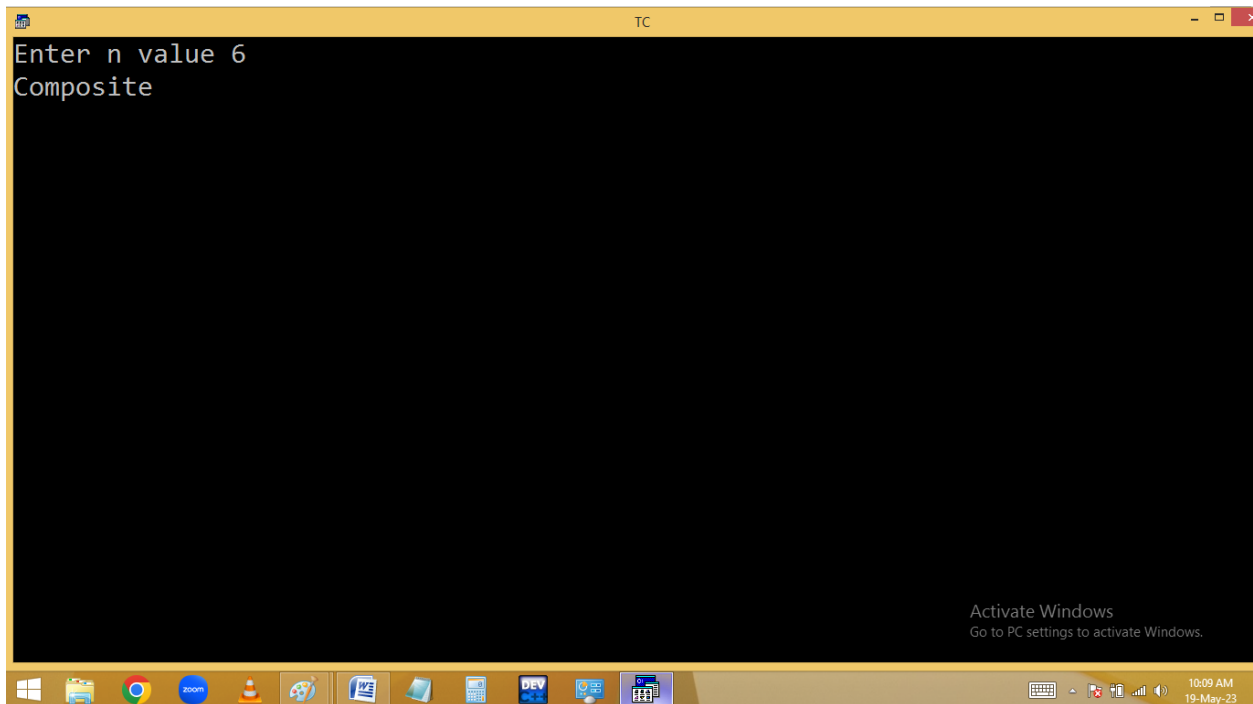
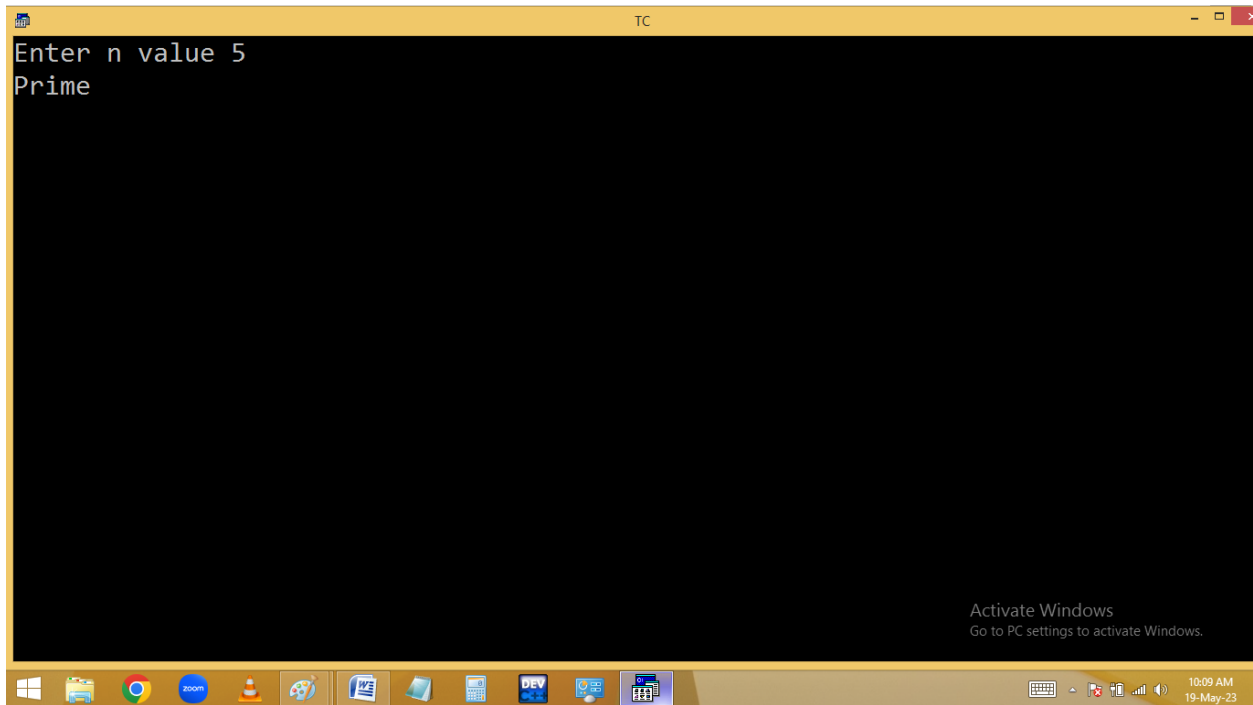
Activate Windows
Go to PC settings to activate Windows.

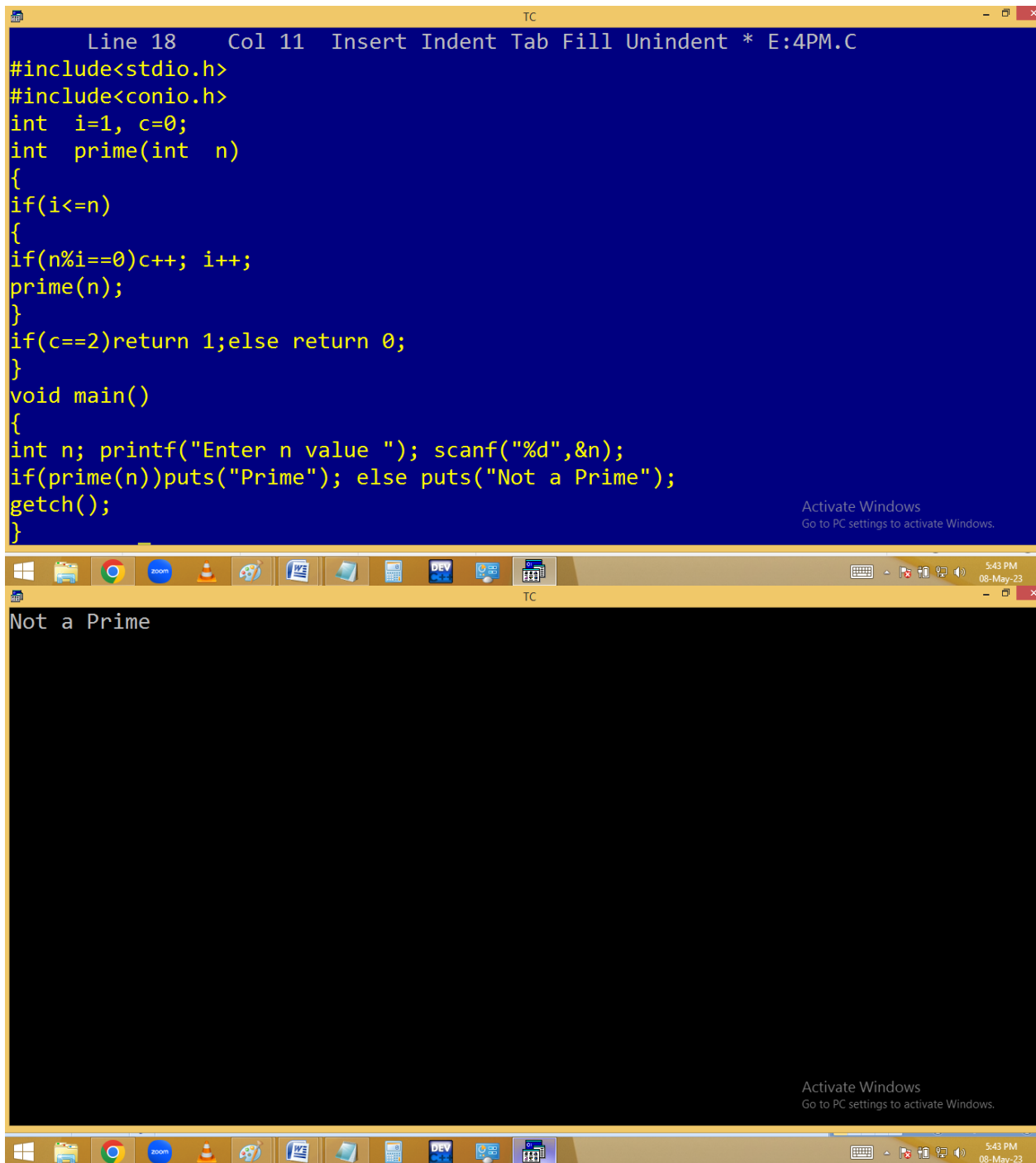
Finding prime using recursion:



```
File Edit Run Compile Project Options Debug Break/watch
Line 7 Col 17 Insert Indent Tab Fill Unindent * E:NONAME.C
#include<stdio.h>
#include<conio.h>
int prime(int n, int i) /* fun def */
{
    if(i==1) return 1;
    else if(n%i==0) return 0;
    else prime(n,--i);
}
void main()
{
    int n;
    clrscr();
    printf("Enter n value "); scanf("%d",&n);
    puts(prime(n, n/2)?"Prime":"Composite");
    getch();
}
```

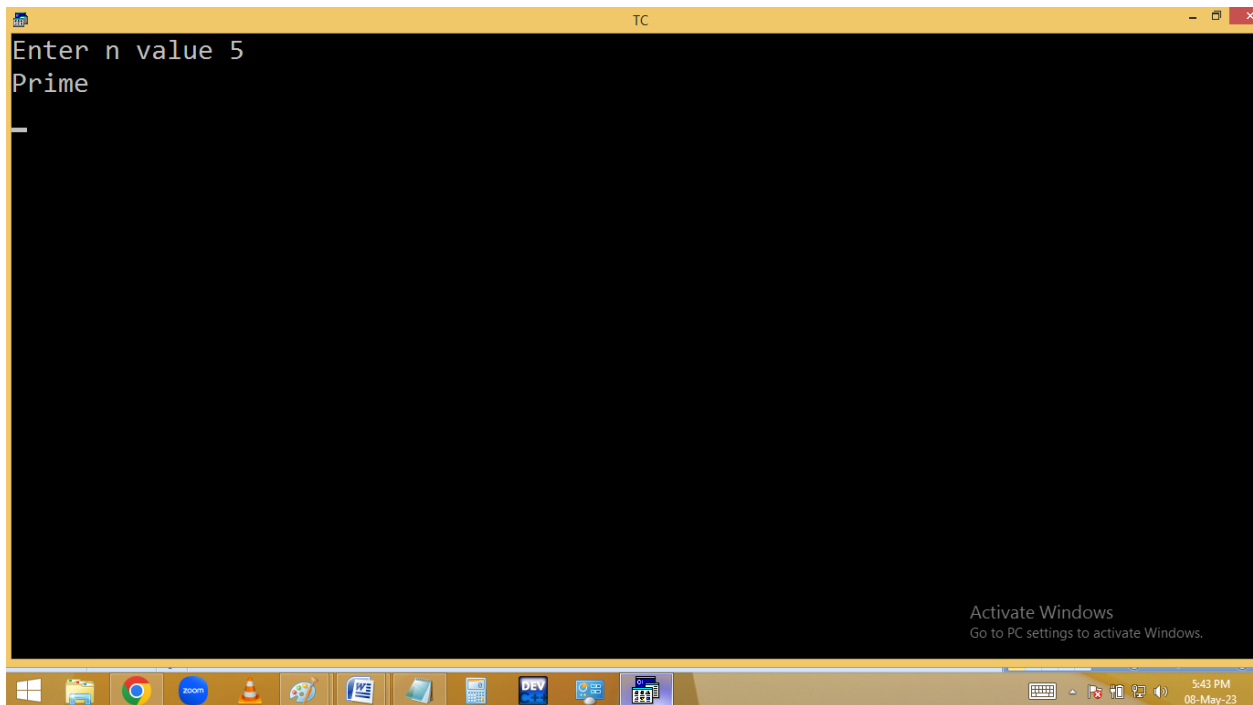
Activate Windows
Go to PC settings to activate Windows.





```
Line 18 Col 11 Insert Indent Tab Fill Unindent * E:4PM.C
#include<stdio.h>
#include<conio.h>
int i=1, c=0;
int prime(int n)
{
if(i<=n)
{
if(n%i==0)c++; i++;
prime(n);
}
if(c==2)return 1;else return 0;
}
void main()
{
int n; printf("Enter n value "); scanf("%d",&n);
if(prime(n))puts("Prime"); else puts("Not a Prime");
getch();
}
```

Not a Prime



```
Enter n value 5
Prime
```

COMMAND LINE ARGUMENTS

It is the process of sending arguments to `main()` from command prompt.

It is useful, in designing applications for dos/command based environment.

It allows to create new commands for CUI[Character User Interface] environments.

`main()` by default having 2 arguments.

1. argc
2. argv

argc is an integer, which counts the no of arguments entered at command prompt, including filename.

argv is a string array(pointer), which stores the arguments entered at command prompt.

The Default no of argument counter is 1.

Eg: Finding n numbers sum using command line arguments:

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<stdlib.h>
```

```
void main(int argc, char *argv[])
```

```
{
```

```
int i, s=0;
```

```
clrscr();  
for(i=1;i< argc ;i++) s = s + atoi(argv[i]);  
printf("Sum = %d", s);  
getch();  
}
```

Save file [eg: sum.c] , compile and ctrl+f9

Goto command prompt.

C:\TC> sum 10 20 30

Sum = 60

Eg:

Finding factorial using command line arguments

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<stdlib.h>
```



```
void main(int argc, char *argv[])  
{  
    long int f=1;  
    int n=atoi(argv[1]);  
    clrscr();  
    while(n>=1) f=f*n--;  
    printf("Factorial = %ld", f);  
    getch();  
}
```

Save file [Eg: fact.c], ctrl+f9

C:\TC> fact 5

Factorial = 120

Eg:

**/*Finding string length using command line
argument*/**

```
#include<stdio.h>

#include<conio.h>

void main(int argc, char *argv[ ])
{
    int i;
    for(i=0;argv[1][ i ]!='\0';i++)
    {
        printf("%-2c",argv[1][ i ]);
    }

    printf(" Length = %d", i);
}
```

Save the file [eg: len.c]

compile [alt+F9], run [ctrl+f9]

goto command prompt.

C:\TC>len kishore

k i s h o r e length = 7